CynLr Line-Scan Scanner Pipeline

A high-performance asynchronous DSP pipeline designed for industrial line-scan camera processing. This system implements a sliding-window Gaussian filter and branchless thresholding, optimized for low-latency hardware verification.

Key Features

Look-ahead Filtering: Implements a symmetric 9-tap Gaussian filter by maintaining a 4-element future queue (`lookahead_`) and a 4-element history buffer. This ensures the filter is always centered on the current pixel.

Memory Efficiency: Adheres to the $\le m$ budget by storing only 8 additional bytes per stream (4 past, 4 future), regardless of the total image width.

Branchless Thresholding: Uses platform-specific inline assembly (**ARM64/x86_64**) for the decision logic. By avoiding standard `if` statements, it prevents CPU pipeline stalls caused by branch mispredictions in noisy sensor data.

Manual Loop Unrolling: The 9-tap convolution is manually unrolled to maximize **Instruction Level Parallelism (ILP)**, ensuring processing stays well under the 100ns/pixel target.

Dynamic Link Architecture: Core logic is compiled into `libcynlr`, separating the DSP engine from the CLI and visualization tools.

Prerequisites

Compiler: `Clang++` (supporting C++17)

Build System: `Make`

Python: 3.8+ (for visualization)

Libraries:

C++: `pthread`

Python: `pandas` , `matplotlib`

Build Instructions

The Makefile handles platform detection and dynamic RPATH settings for macOS, Linux, and Windows.

```
# Build the Shared Library, Scanner Executable, and Unit Tests
make clean && make
```

Running the Scanner

1. Hardware Trace Mode (CSV Input)

Process pre-recorded sensor data and verify logic:

```
make run_csv CSV=data/sample.csv TV=100
```

2. Random Data Mode (Throughput Test)

Stress-test the pipeline with generated data:

```
make run M=16 TV=127.5 T=1000000
```

3. Data Dump Mode (Preparation for Visualization)

Dumps Raw vs. Filtered results to a CSV file:

```
make run_dump CSV=data/sample.csv TV=100
```

Visualization

To verify the signal processing results (Raw Sensor Input vs. Gaussian Smoothed Output):

Install Dependencies:

```
pip install pandas matplotlib
```

Run Visualizer:

```
python3 visualize_pipeline_output.py --csv data/out.csv --tv 100
```

Project Structure

```
.
├── src/                        # Core Implementation
|   ├── main.cpp                # CLI Entry point & Pipeline orchestration
|   ├── Pipeline.cpp            # Block linking and execution logic
|   ├── FilterThresholdBlock.cpp # Gaussian filter & Assembly thresholding
|   └── DataGenerationBlock.cpp  # CSV/Random data producer
|
├── include/                    # Header Files
|   ├── RingBuffer.h            # Thread-safe lock-free communication
|   ├── TimingProfiler.h        # High-resolution hardware timing
|   └── PipelineConfig.h        # Shared constants and CLI settings
|
├── build/                      # Compiled Binaries
|   ├── scanner                 # The dynamic executable
|   └── libcynlr.dylib          # Shared DSP engine library (macOS)
|
├── data/                       # Data Assets
|   ├── sample.csv              # Input sensor trace
|   └── out.csv                 # Generated output dump for analysis
|
├── tests/                      # Unit testing suite
├── images/                     # Documentation assets and plot exports
├── Makefile                    # Platform-aware build system
├── README.md                   # Project documentation
├── CynLr_Design_Overview.pdf   # Architectural specification
├── visualise_filter.py         # Script to plot Gaussian coefficients
└── visualize_pipeline_output.py # Script to plot sensor vs. filtered results
```

Cleanup

```
make clean
```